

Deskripsi & Arsitektur Projek

Tema

Keamanan Penyimpanan Data Cloud Menggunakan Kriptografi Hibrida dan Pembuktian Integritas Data.

Judul

Implementasi Sistem Keamanan Data Cloud Berbasis Dynamic AES Encryption dan Verifikasi Integritas Menggunakan Enhanced Merkle Tree.

Nama Aplikasi

CloudGuard EMT

Desain Arsitektur Sistem:

1. **Fase Input & Hashing Awal:** Pengguna mengunggah dokumen asli melalui antarmuka sistem. File tersebut langsung diproses menggunakan algoritma SHA-256 untuk menghasilkan nilai *hash* (sidik jari). Sistem kemudian melakukan pengecekan deduplikasi; jika *hash* sudah terdaftar, sistem akan memblokir unggahan untuk mencegah redundansi data dan manipulasi rantai blok.
2. **Fase Key Generation (Dynamic AES):** Nilai *hash* file tersebut kemudian dioperasikan dengan logika XOR terhadap nilai *hash* dari blok data sebelumnya (yang ditarik dari riwayat sistem). Operasi ini menciptakan *Dynamic Key* AES-256 yang sangat unik untuk masing-masing file.
3. **Fase Enkripsi:** File asli dienkripsi menggunakan *Dynamic Key* yang baru saja terbentuk, mengubahnya menjadi data acak yang tidak dapat dibaca (*Ciphertext*).
4. **Fase Integritas (Enhanced Merkle Tree):** *Ciphertext* yang terbentuk akan **dipecah menjadi beberapa blok data kecil (*chunks*)**. Masing-masing blok ini di-*hash* untuk dijadikan simpul daun (*leaf nodes*). Simpul-simpul ini disusun membentuk struktur hierarki *Enhanced Merkle Tree* (EMT) untuk menghasilkan satu *Merkle Root* tunggal yang mengunci integritas **file tersebut**.
5. **Fase Storage & Verifikasi:** *Ciphertext* disimpan di dalam *Cloud Storage* terisolasi, sedangkan *Merkle Root* beserta *metadata* dijangkarkan (*anchoring*) ke dalam *database* relasional yang bertindak sebagai Immutable Ledger (Buku Besar Blockchain). Saat pengguna melakukan audit atau meminta unduhan, sistem akan menghitung ulang *hash* dari *Ciphertext* dan memvalidasinya terhadap *Merkle Root*. Jika ada 1 bit data yang berubah, sistem otomatis memblokir dekripsi dan menyatakan rantai data terkorupsi (*Compromised*).

Tech Stack

Frontend (User Interface)

- HTML5, CSS3, & Vanilla JavaScript
- **Framework:** Tailwind CSS
- **Rendering:** Django Template Engine

Backend & Core Engine

- **Bahasa:** Python 3.10+
- **Framework:** Django

Cryptography & Integrity Engine (Trisha & Evan)

- `cryptography`: Eksekusi AES-256-GCM (Enkripsi Dinamis)
- `hashlib`: Kalkulasi SHA-256 (Merkle Tree & Hashing)
- `os.urandom`: Generate Secure Nonce (IV)

Database & Cloud Storage (Iim & Evan)

- **Relational DB (Blockchain & Metadata):** PostgreSQL (Supabase)
- **Public Cloud (Ciphertext Vault):** Supabase Storage
- **DB Connector:** `psycopg2-binary` & `supabase-py` SDK

Deployment & Environment

- **App Hosting:** Render / Railway (Free Tier Deployment)
- **Security:** `python-dotenv` (Manajemen API Key & DB Password)
- **Version Control:** Git & GitHub

Pembagian Tugas

Link GitHub: <https://github.com/trishagarniss/kda-project>

Plan Timeline:  Timeline Task

Nama	Tugas & Tanggung Jawab	Status Pengerjaan
Trisha Garnis W	Modul SHA-256 & Dynamic AES Key Generation. Membangun fondasi logika hashing dan proses pembentukan kunci dinamis (XOR logic).	Completed ▾
Ibrahim Syauqi	Interface & Integrasi Modul. Mengembangkan antarmuka interaktif menggunakan Streamlit/terserah deh dan merangkai semua modul backend menjadi satu alur aplikasi utuh.	In Progress ▾
Jimly Syahbatin	Merkle Tree Construction & Verifikasi. Membangun struktur data pohon hierarkis untuk menghasilkan Merkle Root dan fungsi validasi integritas.	Completed ▾
Evan Hauzal F	Modul Enkripsi/Dekripsi & Simulasi Blockchain. Menulis skrip eksekusi AES-256 untuk memproses file masuk/keluar serta mengatur struktur penyimpanan metadata blok.	Completed ▾

(Draft) Laporan

LAPORAN PROJEK AKHIR
KEAMANAN DATA DAN APLIKASINYA

Implementasi Sistem Keamanan Data Cloud Berbasis
***Dynamic AES Encryption* dan Verifikasi Integritas**
Menggunakan *Enhanced Merkle Tree*



Disusun Oleh:

Trisha Garnis Wahningyun (L0224012)

Ibrahim Syauqi (L0224018)

Jimly Syahbatin (L0224033)

Evan Hauzal Fadhila (L0224041)

PROGRAM STUDI S1 SAINS DATA
FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA
UNIVERSITAS SEBELAS MARET

2026

BAB I

PENDAHULUAN

1.1. Latar Belakang

Penggunaan layanan cloud storage konvensional untuk menyimpan dokumen rahasia dan penting saat ini masih memiliki kerentanan yang tinggi, terutama terhadap ancaman insider threat (penyalahgunaan hak akses oleh pihak internal) dan data breach (peretasan). Pada sistem penyimpanan biasa, pengamanan sangat bergantung pada penyedia layanan. Jika server berhasil diretas, dokumen mentah dapat langsung diunduh dan dibaca dengan bebas oleh pihak yang tidak bertanggung jawab.

Selain masalah kerahasiaan, sistem konvensional umumnya tidak memiliki mekanisme pengawasan untuk mendeteksi silent data tampering (manipulasi data secara senyap). Serangan jenis ini memungkinkan peretas mengubah sebagian kecil informasi (seperti nominal angka atau nilai) tanpa merusak eksistensi file secara keseluruhan, sehingga perubahan tersebut tidak disadari oleh pemilik data. Kerentanan fatal lainnya adalah penggunaan Kunci Enkripsi Utama (Single Master Key) secara terpusat. Jika satu kunci ini bocor (single point of failure), maka seluruh dokumen yang ada di dalam pangkalan data akan terkompromi secara massal.

Oleh karena itu, diperlukan sebuah arsitektur keamanan baru berupa Zero-Trust Document Vault. Sistem ini mengasumsikan bahwa layanan penyimpanan pihak ketiga tidak sepenuhnya dapat dipercaya, sehingga pengamanan data (enkripsi dan penyegelan) harus dilakukan secara mandiri di sisi sistem sebelum data diunggah. Proyek ini dibangun untuk tidak hanya menjamin kerahasiaan (confidentiality), tetapi juga memastikan integritas data (tamper-proof) dan memutus rantai kebocoran melalui manajemen kunci dinamis berkonsep blockchain.

1.2. Rumusan Masalah

1. D
2. F
3. d

1.3. Tujuan

1. E
2. E
3. e

1.4. Batasan Masalah

BAB II

LANDASAN TEORI

[bab ini untuk mensitasi 4 paper dr tiap anggota, file paper di link](#)

2.1. Keamanan Data Cloud & Kriptografi

2.2. Algoritma SHA-256

2.3.

2.4. Dynamic Advanced Encryption Standard (AES)

(Jelaskan konsep XOR antara hash file dan hash blok sebelumnya untuk dynamic key).

2.5. Enhanced Merkle Tree (EMT)

(Jelaskan hierarki pohon hash dan Merkle Root)

Merkle Root merupakan suatu protokol keamanan yang diibaratkan seperti bentuk pohon yang root dan leaf nya saling berhubungan. Jika 1 bit pun diubah dari sisi leaf, maka rootnya pun ikut berubah. Hal ini mengakibatkan gitu gitu lah pokoknya).

BAB III

METODOLOGI DAN PERANCANGAN SISTEM

- 3.1. Arsitektur Sistem**
- 3.2. Perancangan Modul Kriptografi**
- 3.3. Struktur Data Metadata**

BAB IV

IMPLEMENTASI DAN PENGUJIAN

- 4.1. Implementasi Sistem**
- 4.2. Antarmuka Pengguna (User Interface)**
- 4.3. Pengujian Sistem (System Testing)**

BAB V
KESIMPULAN

KESIMPULAN

DAFTAR PUSTAKA

Paper 4 (Dynamic AES + Blockchain) → sebagai core enkripsi

Paper 1 (EMT + SHA-256) → sebagai mekanisme verifikasi integritas

***Paper Deduplication data terenkripsi + dynamic ownership dan Auditing integritas data cloud + user behavior prediction** dijadikan referensi pendukung konsep auditing dan manajemen kepemilikan.*

LAMPIRAN

Tautan Repositori GitHub.

Tabel Log Keterlibatan Anggota (Bukti commit dari GitHub).

Tautan Spreadsheet Task Tracker.

Catatan

- **Hexadecimal**: sistem bilangan berbasis 16 simbol (0–9, A–F), di mana A–F mewakili 10–15.
- **SHA-256**: algoritma hash yang menghasilkan output **256-bit = 64 karakter hex** (karena 1 karakter hex = 4 bit).
- Sifat SHA-256: **Deterministik** → input sama selalu menghasilkan hash yang sama. Perubahan kecil pada input → hasil hash berubah total.
- Hash File + Block Hash Fetch di XOR kan (input beda hasil 1, input sama hasil 0) hitungnya dalam bentuk biner. Jika input sama → hasil tetap sama, jika beda → hasil berubah.
- Blockchain selalu punya **blok awal (blok 0 / genesis block)**. Dibuat manual saat sistem pertama kali dijalankan sebagai fondasi awal.
- Hasil enkripsi disebut **ciphertext**, panjangnya ≈ sama dengan file asli.
- **AES-256** menggunakan kunci (key/password) dari hasil XOR. Untuk dekripsi, sistem membutuhkan key yang sama untuk mengembalikan data asli.
- Data ciphertext dipecah menjadi beberapa **chunk**.
- Setiap chunk di-hash dengan SHA-256 → menjadi **leaf node**.
- Hash digabung bertingkat hingga menghasilkan **Merkle Root (64 karakter hex)**.
- **Merkle Root = ringkasan integritas seluruh file**.
- Yang disimpan di cloud: **ciphertext (bukan file asli)**. Integritas data diverifikasi menggunakan **Merkle Tree**.
- 1 file → 1 ciphertext. 1 ciphertext → 1 Merkle Tree. 1 Merkle Tree → 1 Merkle Root

File Asli → Di-SHA256 → Di-XOR dengan Blok Lama = **Dapat Kunci Rahasia**.

Kunci Rahasia + File Asli → Masuk Mesin AES = **Dapat Teks Acak (Ciphertext)**.

Teks Acak Dipotong-potong → Di-SHA256 berpasang-pasangan naik ke atas = **Dapat Merkle Root**.

Input Awal: File Asli (Bentuk Bytes)

FASE 1: PEMBUATAN KUNCI (KEY GENERATION)

File Asli → SHA-256 → Hash_File

Blockchain → Ambil Blok Terakhir → Prev_Block_Hash

Hash_File ⊕ (XOR) Prev_Block_Hash = Dynamic_AES_Key

Contoh:

Hash_File = a1b2c3...

Prev_Block_Hash = 1f2a3e..

Dynamic_AES_Key: be98fd...

FASE 2: ENKRIPSI DATA (DATA ENCRYPTION)

File Asli + Dynamic_AES_Key + Nonce (IV) → AES-256-GCM = Ciphertext (Data Rahasia)

Contoh:

File_Asli = "Data Rahasia ini..." (*Isi dokumen dalam bentuk teks mentah/bytes*)

Dynamic_AES_Key = be98fd... (*Hasil dari Fase 1*)

Nonce (IV) = 7c8d9e... (*Angka pengacak sekali pakai yang dihasilkan sistem*)

Ciphertext = 0x88ffaa99bb... (*Data yang sudah menjadi sandi acak dan tidak bisa dibaca*)

FASE 3: PENYEGELAN INTEGRITAS (INTEGRITY SEALING)

Ciphertext → Dipotong per 1KB (Chunking) → Chunks

Chunks → Hashing Hierarki (Pohon) = Merkle_Root (Segel 64 Karakter) **Ini bottom-up**

Contoh:

Ciphertext Size: 4 KB

Chunks: Dipotong menjadi 4 bagian (Chunk_1, Chunk_2, Chunk_3, Chunk_4)

Leaf Hash (Daun): Masing-masing di-hash menjadi Hash_A, Hash_B, Hash_C, Hash_D

Branch Hash (Cabang): (Hash_A + Hash_B) = Hash_AB | (Hash_C + Hash_D) = Hash_CD

Merkle_Root: (Hash_AB + Hash_CD) = 3264337ac5252ab63b8dfda1e1f9cd2...

FASE 4: PENYIMPANAN (STORAGE)

Merkle_Root + Nama_File + Prev_Block_Hash → Blockchain = Blok_Baru (Buku Besar)

Ciphertext + Nonce → Cloud Storage = File_Aman_Tersimpan

Contoh blok baru di Blockchain:

```
{
  "Block_ID": 2,
  "Timestamp": "2026-05-06 08:00:00",
  "Nama_File": "laporan_rahasia.pdf",
  "Merkle_Root": "3264337ac5252ab63b8dfda1e1f9cd2...",
  "Prev_Hash": "ee15b63088578781e4e07d0c6057b2a...",
  "Current_Hash": "9f86d081884c7d659a2feaa0c55ad01..."
}
```

Contoh Penyimpanan di Cloud:

File Tersimpan: laporan_rahasia_encrypted.bin

Isi File di Cloud: Hanya berisi Ciphertext (0x88ffaa99bb...) dan Nonce (7c8d9e...). Tanpa Kunci AES dan tanpa Merkle Root, sehingga mustahil untuk diretas.

Yang dipake:

1. Hash File (SHA-256). Dipakai 2 kali: untuk file asli dan untuk *Ciphertext* di EMT
2. Logika XOR > Dynamic AES Key (untuk buat kuncinya)
3. AES-256 (Mode GCM): Mesin pengacak dan pengunci file (menggunakan kunci dari XOR).
4. Enhanced Merkle Tree: Struktur pohon untuk merangkum *hash* dari potongan-potongan *Ciphertext* menjadi satu *Merkle Root*.
5. Simulasi Blockchain: empat untuk mencatat dan merangkai *Merkle Root*, ID File, dan *Timestamp* ke dalam blok-blok yang berkesinambungan. (Ini yang bikin EMT-nya punya nilai historis).
6. Cloud Storage (Simulasi/Lokal): Tempat untuk menampung *Ciphertext* berukuran besar yang sudah dienkripsi oleh AES.

Penentuan Tamper atau Enggaknya

- sistem menentukan file di tamper atau tidak bukan berdasarkan nama file, tapi berdasarkan perbandingan matematis 2 nilai merkle root
- merkle root asli (nilai yg dihitung saat pertama kali di upload lalu disimpan di DB Ledger)
- merkle root hasil audit (nilai yg dihitung ulang sm django setelah download file biner .bin yg ada di supa storage saat tombol verify di klik
- klo file di supa storage meskipun hanya 1 byte (1 char) -> diaudit perhitungan merkle rootnya berubah total (avalanche effect) -> nanti sistem statusnya CORRUPTED
- Nama File SAMA, Isi File SAMA → (Kasus Deduplikasi) paper IIM, status VALID

- Nama File BERBEDA, Isi File SAMA -> krn isi file sama persis hash 256 dan biner terenkripsinya akan tetap sama persis (algo hashing hanya melihat urutan byte isi file, bukan metadata nama file). Sistem akan mendeteksi isi file ini sebagai duplikat melalui *hash*-nya. Nama file yang berbeda hanya akan dicatat di *database* sebagai metadata baru, tetapi file *.bin* di *storage* tidak perlu diunggah ulang ganda. Status VALID

- Nama File SAMA, Isi File BERBEDA -> Karena isinya berbeda (misal ada dokumen kontrak lama dan kontrak baru dengan nama file yang sama), nilai *hash* SHA-256 dan struktur *chunks*-nya akan berubah total. Sistem memperlakukan file ini sebagai **dokumen baru** atau **versi baru** yang akan masuk ke baris *Ledger* berikutnya (Blok baru). Sistem tidak akan menganggapnya sebagai *tamper* selama proses *upload*-nya lewat jalur resmi (antarmuka aplikasi). Status VALID

- Nama File SAMA, Isi File DIUBAH DI CLOUD

Skenario Serangan: File sudah sukses tersimpan di Supabase Storage. Kemudian, ada *insider* (admin Supabase yang nakal) atau *hacker* yang berhasil menembus keamanan Supabase. Mereka membuka file *.bin* tersebut dan mengubah beberapa *byte* datanya (misal mengubah nilai rahasia di dalam file), lalu menyimpannya kembali tanpa lewat aplikasi Django kalian.

Respons Sistem saat di-Verify:

1. Django men-*download* file yang sudah diotak-atik itu.
2. Modul *merkle_tree.py* memotong file tersebut menjadi beberapa *chunks* dan menghitung ulang *Merkle Root*-nya.
3. Hasil *Merkle Root* baru tersebut dibandingkan dengan *Merkle Root* lama yang tersimpan di *database* PostgreSQL.
4. Karena diubah di luar sistem, catatan di *database* tidak ikut berubah. Hasil pencocokan akan **GAGAL** (*Mismatch*).

Status: CORRUPTED / TAMPERED DETECTED!

sistem tidak mendeteksi serangan berdasarkan perubahan nama file atau kemiripan isi saat upload (itu adalah tugas modul deduplikasi data). Sistem kami mendeteksi **Tampering** khusus untuk kasus di mana data yang sudah mengendap di cloud storage dimanipulasi secara ilegal di luar kendali aplikasi Django kami. Tolok ukur mutlak kami adalah **konsistensi nilai Merkle Root** antara yang dikunci di database ledger dengan yang dihitung ulang dari berkas cloud.

Secara arsitektur = MODULAR (bisa bongkar pasang)

Secara eksekusi data = PIPELINE (one way pipeline di [views.py](#))

- Output dari Modul A (Hash) wajib menjadi input untuk Modul B (Dynamic Key).
- Output dari Modul B wajib menjadi input untuk Modul C (AES-GCM).
- Output dari Modul C wajib menjadi input untuk Modul D (Merkle Tree).

Jadi sistem ini menggabungkan keduanya, secara codebase sistem ini sangat modular. Kami memisahkan engine hashing, enkripsi, dan Merkle Tree ke dalam modul Python yang berbeda agar sistem mudah di-maintenance dan di-scale up.

secara aliran eksekusi keamanan (security flow), sistem ini menerapkan Strict Cryptographic Pipeline. Artinya, data dokumen yang di-upload akan melewati lorong pipa modul-modul tersebut secara berurutan dan otomatis. Output dari satu modul menjadi syarat mutlak bagi modul berikutnya. Hal ini kami lakukan untuk menjamin konsep perlindungan *End-to-End* tanpa ada satupun proses keamanan yang bisa di-bypass.

Jika dosen bertanya: "Kalau data ke-3 saya hapus dari database, apakah data ke-4 sampai 8 jadi tidak bisa dibuka/diverifikasi karena rantainya putus?"

"Secara individual, data ke-4 hingga ke-8 **tetap bisa diverifikasi dan dibuka**, Pak/Bu. Hal ini karena nilai **prev_hash** dari Blok 3 sudah direkam secara independen ke dalam metadata Blok 4 saat proses enkripsi awal. Jadi, fungsi XOR dan dekripsi tidak akan gagal."

"Namun, secara arsitektur Ledger, sistem akan mendeteksi adanya **Chain Break (Rantai Terputus)**. Hilangnya Blok 3 membuktikan bahwa telah terjadi pelanggaran manipulasi data (Data Loss) oleh orang dalam yang memiliki akses langsung ke PostgreSQL. Itulah mengapa di skenario nyata, database ledger untuk sistem keamanan seperti ini harus dikonfigurasi dengan aturan **Append-Only** (hanya boleh menambah, dilarang melakukan operasi DELETE sama sekali)."